DEPARTMENT OF SOFTWARE ENGINEERING

VERITAS UNIVERSITY ABUJA

VUA SEN305 Practical Note

Lab Session 2 Client Server Model HTTP and Django MVT Foundation

Semester Project

Enterprise Service Request and Maintenance Platform

Prepared by Bonaventure Anselm

Practical Session Overview

This session produces the first working web slice of the semester project. Students will create a Django project, add the Core application, configure routing, build reusable templates, add static styling, and inspect the HTTP request and response that connect the browser to Django. Every section requires an observable result before you proceed.

Item	Details
Course	VUA-SEN305 Web Application Development Practical
Lab session	2
Duration	3 hours guided laboratory work
Project increment	Public home and about pages for the Service Request Platform
Backend	Python 3.12 and Django 5.2
Frontend for this session	Django templates, HTML and CSS
Database for this session	SQLite development database; MySQL begins in Lab Session 3
Version control	Git feature branch and one verified commit

Practical Outcomes

- Start a Django development server and explain each visible part of the terminal output.
- Create and register a Django application called core.
- Connect project URLs to application URLs without circular imports.
- Render data from a Django view inside an HTML template.
- Build base.html and reuse it through template inheritance.
- Serve a local CSS file through Django static files.
- Create named navigation links that survive later URL changes.
- Inspect GET requests, status codes, headers, HTML responses and missing-page responses.
- Commit the completed project increment to a feature branch.

Required Starting Point

You should have the repository created in Lab Session 1. If the repository does not exist, complete the recovery commands below before beginning. Do not create the Django project inside the virtual-environment directory.

Expected starting structure

```
sen305-service-platform/
├── backend/
├── frontend/
├── docs/
├── infrastructure/
├── .gitignore
└── README.md
```

Laboratory Rules for This Session

- Type each command yourself and read the output before continuing.
- Do not copy an entire completed project from another student.
- When an error occurs, capture the final error line before changing anything.
- Use the exact file paths shown. A correct file in the wrong directory will not be discovered by Django.
- Keep the terminal running the server open; use a second terminal for curl and Git commands.
- Do not commit .venv, __pycache__, db.sqlite3 or secret environment files.

Part A Prepare and Verify the Workspace

Step A1 Open the repository and confirm its location

Open VS Code in the repository root. Then open an integrated terminal and run the appropriate command.

Terminal

```
# macOS or Linux
pwd
ls

# Windows PowerShell
Get-Location
Get-ChildItem
```

Checkpoint result

- The current directory ends with sen305-service-platform.
- The backend, frontend, docs and infrastructure directories are visible.

Step A2 Create the feature branch

Terminal

```
git status
git switch develop
git pull
git switch -c feature/django-mvt-foundation
git branch --show-current
```

Checkpoint result

- The active branch is feature/django-mvt-foundation.
- git status does not report unrelated modified files.

Step A3 Create and activate the virtual environment

Create the environment at the repository root so the backend remains a clean application directory.

Terminal

```
python -m venv .venv

# Windows PowerShell
.venv\Scripts\Activate.ps1

# Windows Command Prompt
.venv\Scripts\activate.bat

# macOS or Linux
source .venv/bin/activate
```

After activation, the terminal prompt should begin with (.venv). Confirm which Python interpreter is active.

Terminal

```
python --version
python -c "import sys; print(sys.executable)"
python -m pip --version
```

Checkpoint result

- Python reports a supported Python 3 version.
- The interpreter path points inside .venv.

- pip is associated with the same virtual environment.

Step A4 Install Django and record the dependency

Terminal

```
python -m pip install --upgrade pip
python -m pip install "Django>=5.2,<5.3"
python -m django --version
python -m pip freeze > backend/requirements.txt
```

Checkpoint result

- Django reports a 5.2.x version.
- backend/requirements.txt contains a Django entry.

Troubleshooting Part A

Symptom	Cause	Correction
python is not recognised	Python is absent from PATH	Use py on Windows or repair the Python installation and enable Add Python to PATH.
Script execution is disabled	PowerShell policy blocks activation	Use Command Prompt activation or apply the institution-approved PowerShell policy.
Django installs globally	Virtual environment is inactive	Activate .venv and verify sys.executable before reinstalling.
requirements.txt not found later	File was created in the wrong directory	Confirm it is backend/requirements.txt from the repository root.

Part B Create the Django Project

Step B1 Create the project inside backend

The final dot tells Django to place manage.py in the current backend directory instead of creating an unnecessary second backend directory.

Terminal

```
cd backend
django-admin startproject config .

# Show the created files
# macOS/Linux
find . -maxdepth 2 -type f

# Windows PowerShell
Get-ChildItem -Recurse -Depth 2 -File
```

Expected structure

```
backend/
├── manage.py
├── requirements.txt
└── config/
    ├── __init__.py
    ├── asgi.py
    ├── settings.py
    ├── urls.py
    └── wsgi.py
```

Step B2 Run Django system checks

Terminal

```
python manage.py check
```

Checkpoint result

- The terminal prints System check identified no issues.
- No module or settings error appears.

Step B3 Apply Django's initial migrations

Terminal

```
python manage.py migrate
```

For this session, Django creates db.sqlite3 only to support the framework's built-in applications. The project will move to MySQL and domain modelling in Lab Session 3.

Checkpoint result

- Migration lines end in OK.
- backend/db.sqlite3 is created.

Step B4 Start the development server

Terminal 1

```
python manage.py runserver
```

Open http://127.0.0.1:8000/ in a browser. Keep this terminal open. The server should display a line similar to Starting development server at http://127.0.0.1:8000/.

Checkpoint result

- The Django installation success page appears.
- Refreshing the browser creates a new GET / HTTP/1.1 200 entry in the terminal.

Checkpoint B First Request Observation

Observation	Student record
Browser URL	_____
Request method from server log	_____
Status code	_____
Date and time	_____

Part C Create and Register the Core Application

Step C1 Create the application

Stop the server with Ctrl+C, create the application, and inspect the files Django generates.

Terminal

```
python manage.py startapp core
```

Created application structure

```
core/
├── migrations/
│   └── __init__.py
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── tests.py
└── views.py
```

Step C2 Register core in installed applications

Open backend/config/settings.py. Add the CoreConfig entry to INSTALLED_APPS. Preserve the existing Django applications.

backend/config/settings.py

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "core.apps.CoreConfig",
]
```

Step C3 Verify registration

Terminal

```
python manage.py check
```

Checkpoint result

- No issue is reported.
- A spelling mistake in core.apps.CoreConfig would now be detected.

Step C4 Create the template and static directories

Terminal

```
# macOS/Linux
mkdir -p core/templates/core core/static/core/css

# Windows PowerShell
New-Item -ItemType Directory -Force core/templates/core
New-Item -ItemType Directory -Force core/static/core/css
```

Required application layout

```
core/
```

```
├── static/
│   └── core/
│       └── css/
├── templates/
│   └── core/
├── urls.py             # create in Part D
└── views.py
```

Why the Repeated core Directory Matters in Practice

The inner core directories namespace the application's templates and static files. This prevents a later application from accidentally replacing a file with the same name. For example, core/templates/core/home.html and accounts/templates/accounts/home.html can coexist safely.

Part D Build the First Request Route

Step D1 Create the home view

backend/core/views.py

```
from django.shortcuts import render


def home(request):
    context = {
        "platform_name": "VUA ServiceDesk",
        "headline": "Report and track maintenance requests",
        "active_requests": 0,
    }
    return render(request, "core/home.html", context)
```

Step D2 Create application routes

Create backend/core/urls.py. The app_name value creates a URL namespace. The name value gives the route a stable identifier for templates and redirects.

backend/core/urls.py

```
from django.urls import path

from . import views


app_name = "core"

urlpatterns = [
    path("", views.home, name="home"),
]
```

Step D3 Include the application routes in the project

backend/config/urls.py

```
from django.contrib import admin
from django.urls import include, path


urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("core.urls")),
]
```

Step D4 Run the server and observe the intentional template error

Terminal 1

```
python manage.py runserver
```

Visit http://127.0.0.1:8000/. At this moment, TemplateDoesNotExist is expected because the view and route work, but home.html has not yet been created. Read the exception name, requested template path and traceback location. Do not change the route to hide the error.

Checkpoint result

- Django reaches core.views.home.
- The diagnostic page names core/home.html as the missing template.

Request Path Built So Far

Sequence	Component	Action
1	Browser	Sends GET /
2	config/urls.py	Delegates the empty path to core.urls
3	core/urls.py	Matches the empty path and calls views.home
4	core/views.py	Creates context and requests core/home.html
5	Template engine	Currently reports that home.html is missing

Part E Create Reusable Templates

Step E1 Create the base template

backend/core/templates/core/base.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}VUA ServiceDesk{% endblock %}</title>
    <link rel="stylesheet" href="{% static 'core/css/styles.css' %}">
</head>
<body>
    <header class="site-header">
        <div class="container nav-row">
            <a class="brand" href="{% url 'core:home' %}">VUA ServiceDesk</a>
            <nav aria-label="Primary navigation">
                <a href="{% url 'core:home' %}">Home</a>
                <a href="{% url 'core:about' %}">About</a>
            </nav>
        </div>
    </header>

    <main class="container">
        {% block content %}{% endblock %}
    </main>

    <footer class="site-footer">
        <div class="container">VUA-SEN305 practical project</div>
    </footer>
</body>
</html>
```

The About URL does not exist yet, so Django will report NoReverseMatch until Part F. This is another useful controlled error: it proves that Django validates named URLs while rendering templates.

Step E2 Create the home template

backend/core/templates/core/home.html

```
{% extends "core/base.html" %}

{% block title %}Home | VUA ServiceDesk{% endblock %}

{% block content %}
<section class="hero">
    <p class="eyebrow">Enterprise Service Request Platform</p>
    <h1>{{ headline }}</h1>
    <p>
        Submit a maintenance request, follow its progress and receive
        clear updates from the assigned team.
    </p>
    <a class="button" href="#request-preview">View request workflow</a>
</section>

<section id="request-preview" class="status-panel">
    <h2>Platform status</h2>
    <p>Active requests in this training environment:</p>
    <strong>{{ active_requests }}</strong>
</section>
{% endblock %}
```

Template Data Check

Template expression	Source	Expected browser value
{{ headline }}	home view context	Report and track maintenance requests
{{ active_requests }}	home view context	0
{% url 'core:home' %}	named URL resolver	/
{% static 'core/css/styles.css' %}	static file resolver	/static/core/css/styles.css

Part F Add the About Page and Named Navigation

Step F1 Add the about view

Append to backend/core/views.py

```
def about(request):
    project_roles = [
        "Requester",
        "Coordinator",
        "Technician",
        "Administrator",
    ]
    return render(
        request,
        "core/about.html",
        {"project_roles": project_roles},
    )
```

Step F2 Add the named route

Update backend/core/urls.py

```
urlpatterns = [
    path("", views.home, name="home"),
    path("about/", views.about, name="about"),
]
```

Step F3 Create the about template

backend/core/templates/core/about.html

```
{% extends "core/base.html" %}

{% block title %}About | VUA ServiceDesk{% endblock %}

{% block content %}
<section class="page-heading">
    <p class="eyebrow">About the semester project</p>
    <h1>One platform for reporting and resolving service needs</h1>
    <p>
        The project will grow each week from these public pages into a
        secure enterprise application with workflows, APIs and reporting.
    </p>
</section>

<section>
    <h2>System roles</h2>
    <ul class="role-list">
        {% for role in project_roles %}
            <li>{{ role }}</li>
        {% empty %}
            <li>No roles have been defined.</li>
        {% endfor %}
    </ul>
</section>
{% endblock %}
```

Step F4 Test both named links

Refresh the browser. Select Home and About repeatedly. Then enter /about/ directly in the address bar. Observe that the URL changes and that the base header and footer remain identical on both pages.

Checkpoint result

- GET / returns 200.
- GET /about/ returns 200.
- Both navigation links work without hard-coded href paths.
- All four project roles appear on the About page.

Part G Add Static Styling

Step G1 Create the stylesheet

backend/core/static/core/css/styles.css

```
:root {
    --ink: #18212f;
    --muted: #5d6776;
    --surface: #f4f7fb;
    --brand: #17365d;
    --brand-dark: #0f2746;
    --line: #dbe2ea;
}

* { box-sizing: border-box; }

body {
    margin: 0;
    color: var(--ink);
    background: #ffffff;
    font-family: Arial, Helvetica, sans-serif;
    line-height: 1.6;
}

.container {
    width: min(1100px, calc(100% - 2rem));
    margin-inline: auto;
}

.site-header {
    border-bottom: 1px solid var(--line);
    background: #ffffff;
}

.nav-row {
    min-height: 72px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 1rem;
}

.brand {
    color: var(--brand);
    font-size: 1.1rem;
    font-weight: 700;
    text-decoration: none;
}

nav { display: flex; gap: 1rem; }
nav a { color: var(--ink); text-decoration: none; }
nav a:hover, nav a:focus { color: var(--brand); text-decoration: underline; }

.hero, .page-heading { padding: 5rem 0 3rem; max-width: 760px; }
.hero h1, .page-heading h1 { font-size: clamp(2rem, 5vw, 3.5rem); line-height: 1.1; }
.eyebrow { color: var(--brand); font-weight: 700; text-transform: uppercase; letter-spacing: .08em; }

.button {
    display: inline-block;
    margin-top: 1rem;
    padding: .75rem 1rem;
    border-radius: .4rem;
    color: #ffffff;
    background: var(--brand);
    text-decoration: none;
}
.button:hover, .button:focus { background: var(--brand-dark); }
```

```
.status-panel {
    margin-bottom: 4rem;
    padding: 1.5rem;
    border: 1px solid var(--line);
    border-radius: .6rem;
    background: var(--surface);
}

.role-list { display: grid; gap: .75rem; padding-left: 1.25rem; }
.site-footer { margin-top: 4rem; padding: 1.5rem 0; border-top: 1px solid var(--line); color: var(--
muted); }

@media (max-width: 560px) {
    .nav-row { align-items: flex-start; flex-direction: column; padding: 1rem 0; }
}
```

Step G2 Verify static-file delivery

Refresh with Ctrl+F5 to bypass the browser cache. Then open the stylesheet URL directly.

Browser

```
http://127.0.0.1:8000/static/core/css/styles.css
```

Checkpoint result

- The page uses the blue navigation and styled sections.
- The direct stylesheet URL returns CSS text rather than 404.
- The layout stacks correctly when the browser width is reduced.

Static File Failure Check

Symptom	Check
Page remains unstyled	Confirm {% load static %} is the first line of base.html.
404 for stylesheet	Confirm core/static/core/css/styles.css has the exact spelling and nesting.
Template syntax error	Confirm the link uses {% static 'core/css/styles.css' %}.
Old CSS remains visible	Use Ctrl+F5 or disable cache in browser developer tools.

Part H Inspect HTTP in the Browser

Step H1 Open browser developer tools

Press F12 or Ctrl+Shift+I, select Network, enable Preserve log if available, and reload the home page.

Step H2 Inspect the document request

Select the request whose name is 127.0.0.1 or /. Record the following values from Headers and Timing.

Field	Observed value
Request URL	_____
Request method	_____
Status code	_____

Remote address	_____
Content-Type response header	_____
Response size	_____
Total request time	_____

Step H3 Inspect the stylesheet request

Select styles.css in the Network list. Verify that it is a separate HTTP request and record its status and content type.

Field	Observed value
Status code	_____
Content-Type	_____
Initiator	_____

Step H4 Generate and inspect a missing-page response

Visit http://127.0.0.1:8000/does-not-exist/. Inspect the Network entry and server terminal. Record the status code. Return to the home page afterwards.

Checkpoint result

- The home document response is 200.
- The missing page response is 404.
- HTML and CSS have different Content-Type headers.
- The browser performs separate requests for the document and stylesheet.

Part I Inspect HTTP from the Command Line

Leave runserver active in Terminal 1. Open Terminal 2 at the repository root. The commands below inspect the same server without using the browser interface.

Step I1 Request response headers

Terminal 2

```
curl -I http://127.0.0.1:8000/
curl -I http://127.0.0.1:8000/about/
curl -I http://127.0.0.1:8000/does-not-exist/
```

On Windows PowerShell, curl may be an alias. Use curl.exe explicitly if the output does not resemble HTTP headers.

Step I2 Request headers and body with verbose output

Terminal 2

```
curl -v http://127.0.0.1:8000/about/
```

In the verbose result, lines beginning with > are request data sent by curl. Lines beginning with < are response data returned by Django.

Step I3 Save an HTML response and inspect it

Terminal 2

```
curl http://127.0.0.1:8000/ -o home-response.html

# macOS/Linux
head -n 20 home-response.html

# Windows PowerShell
Get-Content home-response.html -TotalCount 20
```

HTTP Evidence Table

Request	Expected status	Actual status	Meaning
GET /	200		Home resource returned successfully
GET /about/	200		About resource returned successfully
GET /static/core/css/styles.css	200		Static resource returned successfully
GET /does-not-exist/	404		No URL pattern matched the requested path

Required Student Explanation

Using the observed output rather than memorised definitions, write four sentences that describe what the client sent, how Django selected a view, how the view selected a template, and what the server returned.

Part J Verify the Complete MVT Flow

Practical Trace Home Page

Stage	File or tool	Evidence to point out during demonstration
Client request	Browser Network panel	GET request to /
Project routing	config/urls.py	The empty path delegates to core.urls
Application routing	core/urls.py	The empty path maps to views.home and is named core:home
View execution	core/views.py	home creates headline and active_requests values
Template rendering	core/home.html	Template extends base.html and prints context values
Reusable layout	core/base.html	Header, navigation, stylesheet and footer are shared
Static request	styles.css and Network panel	CSS is returned through a second HTTP request
Server response	Network panel and terminal	HTML document returns status 200

Practical Trace About Page

Repeat the trace for /about/. During the demonstration, identify the loop that converts project_roles from a Python list into four HTML list items. Temporarily add a fifth role in the view, refresh, observe the output, and then remove the temporary role.

Deliberate Break and Repair Exercise

Complete each controlled failure one at a time. Restore the correct code and run python manage.py check before moving to the next row.

Change	Expected failure	Evidence recorded
Rename core/home.html to core/home2.html	TemplateDoesNotExist	Exception name and requested path
Change core:about to core:about_us in base.html	NoReverseMatch	Missing URL name
Change views.home to views.hom in core/urls.py	AttributeError during URL loading	Incorrect attribute name
Change styles.css to style.css only in base.html	CSS request returns 404	Network status and unstyled page

Final Verification Commands

Terminal

```
python manage.py check
python manage.py test
git status
```

Part K Complete the Guided In Lab Exercise

Extend the same Core application. Do not create another Django project or application for this exercise.

Task 1 Add a Contact Page

1. Create a contact view in core/views.py.
2. Pass department_name, email and office_hours through a context dictionary.
3. Add path("contact/", views.contact, name="contact") to core/urls.py.
4. Create core/contact.html that extends core/base.html.
5. Add a Contact link to the shared navigation using {% url 'core:contact' %}.

Task 2 Add a Workflow Page

Create /workflow/ and display the following four stages from a Python list supplied by the view: Request submitted, Request triaged, Technician assigned, Request resolved. Render the stages with a template for loop. Do not type four repeated HTML blocks manually.

Task 3 Add Active Navigation

Use request.resolver_match.url_name in base.html to add aria-current="page" to the active navigation link. Apply a visible CSS style to the active link. Demonstrate the change on Home, About and Contact.

Pattern to adapt

```
<a href="{% url 'core:home' %}"
   {% if request.resolver_match.url_name == 'home' %}aria-current="page"{% endif %}>
    Home
</a>
```

Task 4 Add a Custom 404 Page for DEBUG False

Create a simple 404.html template at the appropriate project-level template location, configure the template directory, and test it only after saving work. Do not expose a public server. Restore DEBUG=True for normal laboratory development after capturing evidence.

Exercise Acceptance Criteria

Criterion	Pass condition
Routes	/, /about/, /contact/ and /workflow/ return 200
Reuse	Every page extends the same base.html
Dynamic content	Contact values and workflow stages originate in Python view context
Navigation	All navigation links use named URLs
Accessibility	The active page has aria-current="page" and keyboard-visible focus
Static files	CSS returns 200 and works at mobile width
Missing route	A nonexistent URL returns 404
System check	python manage.py check reports no issues

Part L Version Control and Submission

Step L1 Update gitignore before staging files

Repository root .gitignore

```
.venv/
__pycache__/
*.py[cod]
backend/db.sqlite3
.env
.env.*
!.env.example
.vscode/
.idea/
node_modules/
```

Step L2 Review the change set

Terminal

```
cd ..
git status
git diff -- backend/config backend/core backend/requirements.txt .gitignore
```

Read the diff. Confirm that no password, unrelated file, database file or virtual-environment file is staged.

Step L3 Stage and commit the increment

Terminal

```
git add .gitignore backend/config backend/core backend/requirements.txt
git status
git commit -m "feat: establish Django MVT foundation"
git log -1 --oneline
```

Step L4 Push the feature branch

Terminal

```
git push -u origin feature/django-mvt-foundation
```

Open a pull request into develop if the class repository workflow is active. The pull request description should state what was built, how it was tested, and any unresolved issue.

Required Submission Evidence

Evidence	What to submit or demonstrate
Repository	Feature branch URL and final commit hash
Home page	Browser view and Network entry showing 200
About or workflow page	Rendered Python list values
Static asset	styles.css request showing 200 and text/css
Missing page	404 response evidence
Terminal	python manage.py check result

Written trace	Four-sentence client-to-response explanation

Quick Troubleshooting Reference

Error or symptom	Most likely cause	First corrective action
No module named django	Wrong interpreter or inactive environment	Activate .venv and verify sys.executable.
manage.py not found	Terminal is outside backend	Change into backend and list files.
Port is already in use	Another server process is active	Stop it or run python manage.py runserver 8001.
TemplateDoesNotExist	Wrong directory or template name	Verify core/templates/core/<name>.html.
NoReverseMatch	Named URL is absent or misspelled	Compare app_name, path name and template url tag.
404 for an intended page	No URL pattern matches the path	Inspect config/urls.py and core/urls.py.
CSS returns 404	Static path is wrong	Verify core/static/core/css/styles.css.
Changes do not appear	File unsaved or browser cache	Save the file, check terminal reload and use Ctrl+F5.
Circular import	urls.py imports config or views incorrectly	Use from . import views inside core/urls.py.
DisallowedHost	Host is not permitted	Use 127.0.0.1 locally; do not add a wildcard for convenience.

End of Session Checklist

- The active branch is feature/django-mvt-foundation.
- The virtual environment and database are excluded from Git.
- Django system check passes.
- Home, About, Contact and Workflow pages return 200.
- A missing route returns 404.
- All pages reuse base.html.
- All navigation uses named URLs.
- CSS loads as a separate successful HTTP request.
- The student can trace browser to URLconf to view to template to response.
- The final commit and evidence are recorded.